Principles of Functional Programming

Personal Notes

Mihir Rao

Contents

1 Prologue

Functional programming treats programming as communication: a program must instruct a computer while explaining an idea to the people who read and extend it. Good programs are *descriptive*, *modular*, and *maintainable*. Standard ML (SML) advances these goals with limited hidden interaction and types checked before evaluation.

1.1 State, purity, and evaluation

An imperative computation often changes *state*, the program's stored information at a particular moment. Its result can therefore depend on earlier events; this Python function returns a different value after every call.

```
count = 0
def increment():
    global count
    count += 1
    return count
```

Understanding `increment()` requires the current value of `count`; one part of the program can change what another part observes.

Definition 1.1 [*Pure function*]. A function is *pure* when it has no observable side effects and always produces the same output from the same inputs.

Functional programming prefers pure components for pure problems. It describes how expressions reduce to values instead of prescribing updates to a shared world, allowing independent expressions to be reasoned about separately.

> *Note.* Purity does not forbid interaction with the outside world. It asks that state be introduced deliberately and isolated when a computation does not require it.

The course organizes this perspective around three recurring principles:

1. **Recursive problems, recursive solutions.** Lists, trees, and other recursively defined structures naturally suggest recursive programs.
2. **Programmatic thinking is mathematical thinking.** Evaluation supports algebraic reasoning, correctness proofs, and complexity analysis.
3. **Types guide structure.** Types document interfaces and rule out behaviors that do not fit those interfaces.

1.2 Expressions and values

In SML, computation is evaluation. The basic object being evaluated is an expression.

Definition 1.2 [*Expression*]. An *expression* is a building block of an SML program. It may evaluate to a value, raise an exception, or continue evaluating forever.

Definition 1.3 [*Value*]. A *value* is a final result that cannot be simplified further. Constants such as `2`, `"hi"`, and `true` are values; `2 + 2` is an expression that is not yet a value.

Write $e \Longrightarrow e'$ when expression e takes one evaluation step to e'. Its reflexive, transitive closure is written $\Longrightarrow^*$, meaning zero or more steps. For example,

$$(2 + 3) \cdot 4 \Longrightarrow 5 \cdot 4 \Longrightarrow 20,$$

so $(2 + 3) \cdot 4 \Longrightarrow^* 20$. This sequence is a *computation trace*; the expression evaluates to 20.

Not every well-formed-looking expression yields a value. Evaluating `1 div 0` raises an exception, and a recursive computation with no base case may run forever. Thus evaluation has three possible behaviors:

1. reduction to a value,
2. an exception, or
3. nontermination.

1.3 Types predict permitted behavior

The expression `"hi" ^ " there"` concatenates two strings, whereas `"1" ^ 50` gives a string operator an integer operand. SML rejects the mismatch instead of inventing a conversion.

Definition 1.4 [*Type*]. A *type* specifies the permitted behavior of a piece of code. The judgment $e : t$ states that expression e has type t.

Typing follows compositional rules. A simplified rule for integer addition is

$$\frac{e_1 : \texttt{int} \qquad e_2 : \texttt{int}}{e_1 + e_2 : \texttt{int}}.$$

This proves $1 + (2 + 3) : \texttt{int}$ by typing each literal, then applying the rule to the inner and outer additions. String concatenation instead requires two `string` operands; because $50 : \texttt{int}$, `"1" ^ 50` is ill-typed.

Definition: A program *type-checks*, or is *well-typed*, when it obeys the typing rules. A program that

violates them is *ill-typed*.

SML is *statically typed*: it checks types before evaluation and does not run ill-typed programs. An expression of type `int` must produce an integer if it produces a value, but it may raise an exception or run forever.

1.4 Declarations and function application

Declarations associate names with typed program components. A function declaration has a name, typed parameters, a declared result type, and a body:

```
fun double (n : int) : int = n + n
```

Function application is written by adjacency, so applying `double` to `2` is written `double 2`, and

$$\texttt{double 2} \Longrightarrow 4.$$

The type of `double` is `int -> int`: it accepts an integer and returns an integer. The annotations are checked rather than trusted. Claiming a `string` result while retaining `n + n` would make the body type disagree with the annotation, so SML would reject the declaration.

More generally, function application obeys the rule

$$\frac{e_1 : t_1 \to t_2 \qquad e_2 : t_1}{e_1\ e_2 : t_2}.$$

The argument type must match the function's input type, and the application has the function's output type.

A value declaration uses `val` and has no parameter list:

```
val favoriteCourseNumber : int = 150
```

Together, expressions, values, evaluation, types, and declarations form the core vocabulary for reasoning about SML programs. Later lectures refine this model with binding, scope, recursion, and richer forms of data.

Source: Brandon Wu, Carnegie Mellon University 15-150, Principles of Functional Programming, *Summer 2023, Lecture 1: "Prologue," official slides and course materials,* https://brandonspark.github.io/prologue/lecture01.pdf *and* https://brandonspark.github.io/150/ *(accessed August 13, 2026).*